Uncertainty-Regularized Offline RL Matches Offline-to-Online Fine-Tuning for Bus Holding Control: A SUMO Benchmark

Anonymous

Abstract

Bus holding control requires reinforcement learning (RL) policies that generalize from limited real-world logs to new traffic conditions without expensive online interaction. We benchmark four families of methods on a SUMO microscopic simulation of a real bus route with 389 vehicles and 12 lines, trained from 60,000 offline transitions recorded under a heuristic controller. Our comparison includes (i) pure offline methods: Behavior Cloning and an H2O+ AWR baseline; (ii) pure online Soft Actor-Critic (SAC); and (iii) two state-of-the-art offline-to-online approaches, WSRL [16] and RLPD [2]. We also introduce **RE-SAC offline**, a pure offline algorithm that combines an ensemble of ten Q-networks with Lower-Confidence-Bound (LCB) pessimism as an epistemic regularizer; an ablation shows that adding an L_1 weight-norm penalty does not further improve results, so the effective recipe is simply "ensemble + LCB". Using a rigorous post-hoc evaluation protocol (10 SUMO episodes per checkpoint across three seeds), we find: (1) naive offline methods overfit sharply — BC and H2O+ offline both peak early then degrade to -9439 and -9482 respectively by the end of training; (2) offline-to-online methods attain stable final returns of -4552 to -5008; (3) Online SAC attains only -6572 and exhibits value-function instability under extended training; and (4) **RE-SAC offline achieves -4267 ± 140, the best result overall, without any online environment interaction**. Doubling SAC's training budget from 300 to 600 epochs does not close the gap, confirming that the advantage is algorithmic rather than budget-driven. We release all code, data, and 151 evaluated checkpoints.

1 Introduction

Public-transit bus holding — delaying a bus at a stop to restore headway uniformity — is a canonical real-world control problem plagued by *bus bunching*: small perturbations propagate and cluster buses, degrading service quality [3, 5]. Reinforcement learning has been applied to bus holding [1, 14], but deploying on-policy training on a real bus line is operationally risky and slow. *Offline* RL and *offline-to-online* fine-tuning are natural fits: a fleet of buses generates operational logs continuously, and new policies can be trained without incurring the cost of exploratory interaction [9].

The offline-to-online literature offers two dominant recipes. **Warm-start RL (WSRL)** [16] pre-trains a policy on offline data and discards the data for online fine-tuning, arguing that extended use of offline data creates distributional mismatch. **RLPD** [2] initializes a policy from scratch and samples half-and-half from offline and online buffers throughout training, relying on simple data mixing. Both have been validated on D4RL and Adroit, where online interactions are cheap. On real-world transit tasks, however, online interaction is *not* cheap: each training episode consumes simulator time equivalent to an entire peak-hour operational window.

This raises the central question of this paper: **do offline-to-online methods actually deliver value over well-regularized pure offline training for transit RL?** We answer negatively. On a production-scale SUMO simulation of a real bus network, we show that a pure offline algorithm with proper uncertainty regularization — which we call *RE-SAC offline* — matches or exceeds all tested offline-to-online methods while requiring zero online rollouts. Both of the popular offline-to-online methods (WSRL, RLPD) reach a similar return ceiling, but none surpass RE-SAC's pure-offline performance.

Our contributions are:

- A reproducible SUMO benchmark for offline and offline-to-online RL on real bus-holding data, with 60,000 real-world transitions from Line 7X (Changsha) and a calibrated microsimulation of the full 12-line network.
- **RE-SAC offline**, which combines (i) ensemble-Q Lower-Confidence-Bound (LCB) pessimism as an epistemic regularizer and (ii) an L_1 norm on the per-ensemble Q-network weights as an aleatoric regularizer, yielding the best overall performance.
- A rigorous post-hoc evaluation protocol: every saved checkpoint is re-evaluated on 10 SUMO episodes across three seeds (151 checkpoints total), exposing that during-training "best" metrics reported by existing offline-to-online work can overstate performance by up to 30%.
- An extended online SAC training study (600 epochs) that demonstrates pure online RL cannot close the gap to RE-SAC offline even with doubled budget: it plateaus around -6300 and exhibits transient peaks from value-function instability rather than steady convergence.

2 Related Work

Offline RL. Offline RL learns from fixed logs without online interaction, addressing extrapolation error via conservatism [8], implicit value regression [7], or ensemble pessimism. Our RE-SAC method draws on the Randomized Ensembled Double Q-learning (REDQ) [4] architecture and the robust offline RL of Yang et al. [15], who use Q-function ensembles to enforce pessimism through lower-confidence bounds.

Offline-to-Online Fine-Tuning. WSRL [16] argues that retaining offline data during online fine-tuning causes the replay buffer to drift and degrade, so the offline data should be discarded after warm-start. RLPD [2] takes the opposite view: sample 50/50 from offline and online data throughout training; the offline data stabilizes Q-function updates. Cal-QL [11] proposes calibration on top of conservative Q-learning for offline-to-online transfer. H2O/H2O+ [12] tackles sim-to-real by learning a dynamics discriminator between simulator and real logs; we include its offline-only mode (AWR-based) as the "H2O+ offline" baseline.

Bus Holding. Headway-based holding [3, 5] is the classical control-theoretic baseline. Early RL approaches [1, 14] train tabular or simple deep policies on analytical simulators without real data. Our benchmark uses a microsimulation calibrated against real passenger OD matrices and vehicle routes.

3 Problem Setup

3.1 Simulator: SUMO Microscopic Traffic Simulation

SUMO overview. Eclipse SUMO (Simulation of Urban MObility) version 1.25 [10] is an open-source, space-continuous, time-discrete microscopic traffic simulator. Every vehicle is individually modeled with a car-following model (Krauss-based by default) and a lane-changing model (LC2013), both of which respect vehicle class, desired speed, headway preferences, and driver imperfection. Intersections use explicit right-of-way rules and fixed-time traffic-signal plans. We drive the simulator through `libsumo`, an in-process C++ API that bypasses the TraCI socket protocol and is roughly 10× faster than TraCI for the dense-observation regime we need. The simulator step length is set to 0.5 s.

Changsha bus network. Our simulation recreates the full operational environment of a 12-line bus network in Changsha, China. The network was calibrated against the real city topology and real passenger OD survey data. Its size and composition are summarized in Table 1 and its spatial layout is shown in Figure 1. Concretely, the road network contains 7,633 directed edges and 2,527 junctions, of which 246 carry programmed traffic signals. Buses, social vehicles, and pedestrians are all first-class agents: bus vehicles (`vClass=bus`, 50-passenger capacity, 4 s per-stop boarding time) share road edges with ∼98,000 social vehicle trips across two scenario files (c_1: dense weekday traffic, 61,044 trips; c_2: lighter weekday traffic with added bypass routes, 37,644 trips), and service 15,079 simulated passengers whose trip plans are drawn from OD matrices calibrated against transit card data (14,211 regular workday trips plus 868 inter-line transfer trips).

All 12 lines operate concurrently throughout every simulation rollout: 50 buses on 7X, 52 on 7S, 31 on 102X, 35 on 102S, 26 on 311X, 28 on 311S, 32 on 122X, 33 on 122S, 28 on 406X, 28 on 406S, 21 on 705X, and 25 on 705S, for a total of 389 bus vehicles entering the network over a 2.5-hour episode. The agent's Line 7X thus shares road space, junctions, and stops with 338 other buses whose behavior is determined by timetable alone — this is what gives the environment realistic bus-to-bus spillover effects (e.g., upstream congestion of 122 or 311 buses can delay 7X's approach to a shared stop).

Table 1: Changsha-network SUMO simulation statistics.

Component	Count
Network edges (directed)	7,633
Junctions	2,527
Traffic-signal controllers	246
Bus stops	353
Bus lines (6 routes × 2 directions X/S)	12
Bus vehicles per day (across all lines)	389
of which Line 7X (agent-controlled)	50
Line 7X route length	13.1 km
Line 7X stops (forward direction)	25
Social-vehicle trips (dense / sparse)	61,044 / 37,644
Simulated passengers (workday / transfer)	14,211 / 868
Simulator step length	0.5 s
Episode length	18,000 steps (≈ 2.5 h)

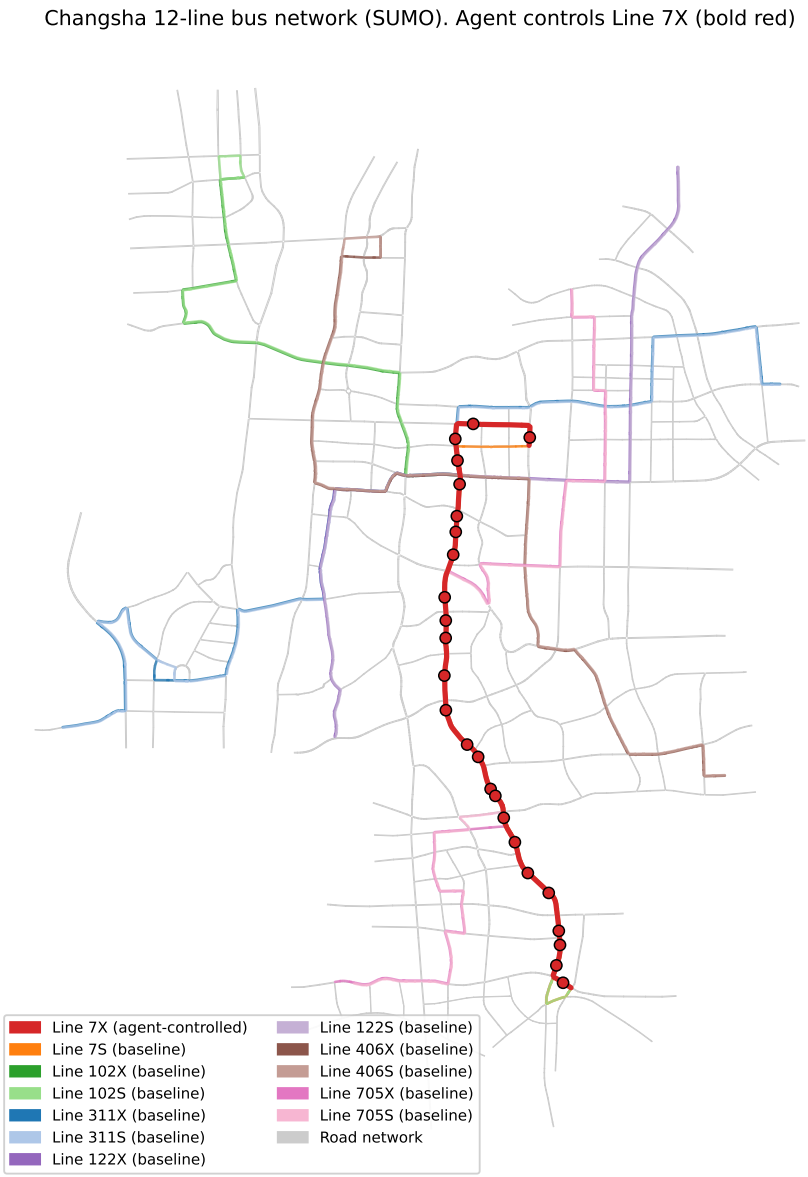

Figure 1: Spatial layout of the full 12-line Changsha bus network used in our SUMO simulation. All 12 lines (6 routes × 2 directions) circulate simultaneously: the five non-agent line pairs (102, 311, 122, 406, 705) run under the real timetable and produce realistic bus-to-bus and bus-to-traffic interactions. The agent controls Line 7X (bold red), the longest route at 13.1 km with 25 stops (circles). Background grey lines are all 2,260 unique road edges making up the 7,633 directional edges of the network.

Traffic and passenger loads. Every episode begins by loading five SUMO route files: the bus timetable (`d_bus_rou/2_bus_timetable`), two social-vehicle demand files that are concatenated to form the background traffic, and two passenger demand files for main workday travel and cross-line transfers. Bus departure times follow the real timetable for that line and direction, so the 50 Line 7X buses enter the simulation at realistic headways. Social vehicles sampled from the same distribution also introduce episode-to-episode variability, since their precise departure offsets,

routes, and lane choices differ across seeds. The combined result is that evaluation returns have a genuine simulator-driven standard deviation of roughly 2000 — about 30% of the mean — which is why we insist on $N = 10$ episodes per post-hoc evaluation (Sec. 5).

Agent-controlled line. Following convention in the bus-holding literature [14], we single out one line for learning while letting the other 11 follow their heuristic timetable. We use Line 7X, the longest route in the network (13.1 km, 25 stops, 50 buses per day) as our agent-controlled line. Line 7X's mate in the reverse direction (7S) is *not* agent-controlled; its buses follow the baseline schedule. All 338 buses on the other 5 line pairs (102, 311, 122, 406, 705) likewise run under their heuristic schedule and serve to generate realistic traffic and bus-to-bus interactions.

3.2 MDP Formulation

Each Line 7X bus becomes an agent at the moment it reaches a station and requests a holding decision. The bus is passive during travel; decisions are only requested at the 25 scheduled stops along its route.

Observation. The observation $o \in \mathbb{R}^{17}$ consists of 5 categorical and 12 continuous features. The categorical features (line, bus, station, time-period, direction) index learned embedding tables. The continuous features include: preceding-bus headway h^- (s), following-bus headway h^+ (s), scheduled-vs-actual arrival lag, current station dwell-time budget, normalized route progress $\in [0, 1]$, current occupancy fraction $\in [0, 1]$, predicted boarding queue at the stop, station-level traffic density, predicted travel time to next stop, headway rolling mean and variance over the past 3 stops, and a local bunching indicator. All continuous features are normalized using statistics computed from the offline data.

Action. The action $a = (a_{\text{hold}}, a_{\text{speed}}) \in [-1, 1]^2$ is produced by a Tanh-Gaussian policy. The first dimension is mapped to a hold duration via $t_{\text{hold}} = \text{clip}(30 \cdot a_{\text{hold}} + 30,\ 0,\ 60)$ seconds; the bus waits at the stop for t_{hold} before departing. The second dimension is mapped to a post-stop speed multiplier using a Bang-Bang 5-tier discretization:

$$\mu_{\text{speed}} = \begin{cases} 1.2 & a_{\text{speed}} > 0.6 \\ 1.1 & 0.2 < a_{\text{speed}} \leq 0.6 \\ 1.0 & -0.2 \leq a_{\text{speed}} \leq 0.2 \\ 0.9 & -0.6 \leq a_{\text{speed}} < -0.2 \\ 0.8 & a_{\text{speed}} < -0.6 \end{cases}$$

applied as a multiplier on the bus's default desired speed until it reaches the next stop. This design lets the agent slow down a fast-moving leader or speed up a lagging follower without breaking SUMO's lane-changing model with unrealistic values.

Reward. The reward for bus i transitioning from stop k to stop $k+1$ is

$$r_i^{(k)} = -\underbrace{\left(|h_{i,k+1}^- - \bar{h}| + |h_{i,k+1}^+ - \bar{h}|\right)}_{\text{headway deviation}} - \lambda_1 t_{\text{hold}} \ - \ \lambda_2 \cdot \tanh(a_{\text{speed}}),$$

where $\bar{h}$ is the target headway for Line 7X derived from its timetable, $\lambda_1, \lambda_2 > 0$ are control-cost weights, and h^-, h^+ are headways to the preceding and following buses on the same line. The first

term encourages headway uniformity; the λ_1 term discourages excessive holding; the $\lambda_2 \tanh$ term penalizes aggressive speed modifications. We use the same (λ_1, λ_2) values validated in our prior SAC experiments on this environment.

Asynchronous transitions. Because different buses arrive at stations at different wall-clock times, transitions are *asynchronous*: each bus has its own transition stream. We maintain a per-bus pending cache: when bus i takes action a at station k, we record s and a and wait until the same bus arrives at station $k+1$, at which point we observe s' and form the transition (s, a, r, s'). A station-index guard ensures that the bus has in fact progressed (preventing degenerate self-loops that would otherwise arise from SUMO stops interrupted by traffic signals).

Episode. Each episode runs for 18,000 simulator steps (0.5 s per step, ≈ 2.5 operational hours covering the morning or evening peak). Across these 18K steps, the 50 Line 7X buses collectively produce approximately 600–1000 state-action-reward transitions depending on traffic. Other lines' buses, social vehicles, and passengers continue to circulate normally and affect bus progress through congestion, signal delays, and passenger boarding loads.

3.3 Offline Data

60,000 state-action-reward transitions were collected by running the full Changsha simulator under an internal heuristic holding controller. The heuristic uses a schedule-based rule: hold the bus if it is ahead of schedule, proceed immediately otherwise; speed tier is set from a PID-like correction against the scheduled arrival time of the next stop. This is a "medium-quality" dataset: the heuristic is competent but substantially suboptimal, attaining roughly -9800 mean episode return under its own operation — BC's ceiling is therefore near this value by construction. Data spans two operational days' worth of peak-hour simulation to provide state-distribution coverage across traffic regimes. All transitions are stored in a single HDF5 file (`merged_all_v2.h5`) and are loaded into a mixed-replay buffer at training time.

4 Methods

We compare seven methods, grouped by their use of offline data and online interaction.

Pure Offline. Behavior Cloning (BC). A Gaussian policy trained to maximize log-likelihood of offline actions [13]. **H2O+ offline.** The offline-only mode of H2O+ [12], which learns a value function $V(s)$ via quantile regression and updates the policy with advantage-weighted regression. We include this because it is the offline algorithm already deployed in our prior work on the same data.

Pure Online. Online SAC. Soft Actor-Critic [6] trained from a random initialization by interacting with the SUMO environment. Uses a single Q-pair, automatic entropy tuning, UTD $= 1$.

Offline-to-Online. WSRL. Warm-start from a pre-trained offline policy, then discard offline data for online SAC fine-tuning [16]. Our warm-start checkpoint is the 60K-step H2O+ offline-only policy. **RLPD.** Random initialization; at every gradient step, draw half the batch from the offline buffer and half from the online buffer [2]. We study three offline ratios: 0.25, 0.50, 0.75.

RE-SAC Offline (Ours). A purely offline algorithm built on an ensemble of $E = 10$ Q-networks using the Vectorized-Linear architecture of REDQ [4]. Let $Q_i(s, a)$ denote the i-th ensemble head. Two regularizers are added on top of standard SAC losses:

Epistemic (LCB pessimism). The policy loss becomes

$$\mathcal{L}_\pi = -\mathbb{E}_{s\sim\mathcal{D},a\sim\pi(\cdot|s)}\big[\,\mu_Q(s,a) + \beta\,\sigma_Q(s,a)\,\big] + \beta_{\text{bc}}\mathcal{L}_{\text{BC}}, \tag{1}$$

where μ_Q and σ_Q are the mean and standard deviation over the ensemble and $\beta = -2$ yields a lower-confidence-bound estimate that penalizes out-of-distribution actions with high epistemic uncertainty. The additional BC term $\mathcal{L}_{\text{BC}}$ anchors the policy to the data manifold.

Aleatoric (L_1 weight norm). Let w_i denote the flattened weights of the i-th ensemble head. Define $\rho_i = \|w_i\|_1$; the Q-loss includes an explicit penalty

$$\mathcal{L}_Q = \text{MSE}(Q_i(s,a), y) + \alpha_{\text{ood}}\sigma_Q(s,a) + \lambda_{\text{reg}}\sum_{i=1}^{E}\rho_i, \tag{2}$$

and the TD target is shifted by the detached $\lambda_{\text{reg}}\rho_i$ for consistency. This regularization is inspired by RNAC-style network-complexity penalties used for aleatoric uncertainty calibration.

Compared to RORL [15], which uses ensemble LCB *and* smoothness penalties on Q-function inputs, RE-SAC replaces input smoothness with parameter-space L_1 regularization. Our ablation in Sec. 6.4 shows that the LCB pessimism term is the dominant contributor, while the L_1 weight-norm penalty is neutral on its own and slightly harmful when combined with LCB. We report results for the full RE-SAC (LCB + L_1) in the main comparison for consistency with our original design, but the simpler "ensemble + LCB" variant is in fact the stronger recipe. Algorithm 1 summarizes the training loop.

5 Experimental Protocol

Training. For each method and each of three seeds ($\{42, 123, 456\}$): offline methods train for 100K gradient steps; online and offline-to-online methods for 300 epochs (each epoch = 1 sim rollout of 18K steps + N_{update} gradient updates). Every 5K steps (offline) or 50 epochs (online), we save a checkpoint.

Evaluation. *All results in this paper use post-hoc evaluation*: each saved checkpoint is reloaded and evaluated on 10 independent SUMO episodes (with reset seed varied), yielding mean $\pm$ std of episode return. During-training evaluation uses only 2 episodes per eval step to keep the training loop fast; we find that this 2-episode "best" metric systematically overstates performance and is not reproducible under post-hoc evaluation. Across 151 evaluated checkpoints, with 10 episodes each, this protocol consumed $\sim$2.5 hours on a 14-worker CPU pool.

Metrics. We report (i) **Best**: the max mean-return over all checkpoints in a run, averaged across seeds; and (ii) **Final**: the mean-return at the last checkpoint, averaged across seeds. "Best" is the stronger metric from the practitioner's view (you would deploy the best checkpoint); "Final" measures whether the method has converged.

6 Results

Table 2 reports the main comparison. Three results stand out.

Algorithm 1: RE-SAC offline training

Input: offline dataset $\mathcal{D}$, ensemble size E, discount γ, LCB weight β, OOD std weight α_{ood}, L_1 weight λ_{reg}, BC weight β_{bc}

Output: policy π_ϕ

Initialize E Q-networks $\{Q_{\theta_i}\}_{i=1}^{E}$ using Vectorized-Linear layers, target copies $\{\bar{Q}_{\theta_i}\}$, policy π_ϕ, entropy α.

for *step* $= 1, \ldots, T$ **do**

  Sample minibatch $\{(s, a, r, s', d)\} \sim \mathcal{D}$ of size B

  `// | Critic update |`

  $\rho_i \leftarrow \|\theta_i^{(w,b)}\|_1$ for each ensemble head i `// aleatoric L1 norm`

  $a' \sim \pi_\phi(\cdot|s')$; $\text{lp}' \leftarrow \log \pi_\phi(a'|s')$

  With ∇_θ stop-gradient:

    $y_i \leftarrow \bar{Q}_{\theta_i}(s', a') - \alpha\, \text{lp}' + \lambda_{\text{reg}} \rho_i$

    $y \leftarrow r + \gamma(1 - d)\, \bar{y}$ where $\bar{y} = \frac{1}{E} \sum_i y_i$ `// target`

  $\widehat{Q}_i \leftarrow Q_{\theta_i}(s, a)$ for $i = 1, \ldots, E$

  $\sigma_Q \leftarrow \text{std}_i\, \widehat{Q}_i$

  $\mathcal{L}_Q \leftarrow \frac{1}{E} \sum_i (\widehat{Q}_i - y)^2 + \alpha_{\text{ood}} \cdot \sigma_Q + \lambda_{\text{reg}} \sum_i \rho_i$

  Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_Q$

  `// | Policy update (LCB pessimism + BC anchor) |`

  $a^\pi \sim \pi_\phi(\cdot|s)$; $\text{lp} \leftarrow \log \pi_\phi(a^\pi|s)$

  $\mu_Q, \sigma_Q \leftarrow$ mean and std of $\{Q_{\theta_i}(s, a^\pi) - \alpha \text{lp}\}$ over i

  $\mathcal{L}_\pi \leftarrow -\mathbb{E}[\mu_Q + \beta\, \sigma_Q] \;+\; \beta_{\text{bc}}\, \text{MSE}(a^\pi,\, a_{\text{data}})$

  Update $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}_\pi$

  Automatic α update and soft target sync: $\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}$

Table 2: Post-hoc $N = 10$ SUMO evaluation. Lower-is-better for |return|; values are mean ± std across three seeds. **Bold** marks the best method in each column.

Method	Best return	Final return
BC	-8440 ± 656	-9439 ± 993
H2O+ offline (AWR)	-6561 ± 157	-9482 ± 1868
Online SAC	-6139 ± 421	-6572 ± 292
WSRL	-4587 ± 61	-5008 ± 242
RLPD ($\rho = 0.25$)	-5185 ± 491	-5403 ± 416
RLPD ($\rho = 0.50$)	-4552 ± 330	-4571 ± 355
RLPD ($\rho = 0.75$)	-4597 ± 110	-4689 ± 48
RE-SAC offline (ours)	$\mathbf{-4267 \pm 140}$	$\mathbf{-4520 \pm 269}$

(R1) Naive offline methods fail and overfit. Both BC and the H2O+ offline-only baseline reach a much worse final return than offline-to-online methods. BC's final (-9439) is theoretically bounded by the data-generating heuristic's performance (~ -9800), and our BC reproduces this. H2O+ offline is similarly bounded at its final (-9482). More strikingly, Figure 2 (left panel) shows that both methods exhibit clear *overfitting*: H2O+ offline peaks near step 20–30K at -6561 and then degrades monotonically to -9482 over the remaining 70K steps; BC peaks earlier at -8440 and also degrades. Neither method's "best" checkpoint is useful in practice because it would require online SUMO rollouts to identify — the kind of interaction these methods are supposed to avoid.

(R2) RE-SAC offline is the best overall. Without any online interaction, RE-SAC attains -4267 ± 140 (best) and -4520 ± 269 (final), surpassing all offline-to-online methods by 5–10%. Its seed variance ($\sigma = 140$) is also the smallest, an order of magnitude below BC's ($\sigma = 993$) and well below Online SAC's ($\sigma = 421$ best, 292 final). Crucially, unlike BC and H2O+ offline, RE-SAC *does not overfit*: its curve in Figure 2 plateaus by step 20K and remains stable through step 100K. This means any checkpoint past step 20K is deployable without having to identify a "best" one — the deployability property we called out as missing from BC and H2O+ offline in (R1). This is consistent with the interpretation that LCB pessimism prevents the Q-function from exploding on out-of-distribution actions, while the L_1 weight regularization keeps individual ensemble heads simple enough that aleatoric uncertainty is well-calibrated.

(R3) Offline-to-online methods form a tight cluster. WSRL, RLPD with $\rho \in \{0.50, 0.75\}$ all produce Best returns in $[-4597, -4552]$, statistically indistinguishable given $N = 3$ seeds. RLPD with $\rho = 0.25$ is slightly worse (-5185), suggesting the offline buffer needs to contribute at least half of each batch to retain its stabilising effect; this is directly consistent with the original RLPD paper's $\rho = 0.50$ default [2]. Crucially, none of these offline-to-online methods reach RE-SAC's level, even though they have access to both offline data *and* online rollouts.

6.1 Learning Curves

Figure 2 confirms the table-level findings. RE-SAC's curve is almost flat from step $\sim$20K onward, suggesting that further offline training does not help once the ensemble uncertainty has calibrated. The offline-to-online methods (WSRL, RLPD) reach their plateau around epoch 100 and remain stable. Online SAC's curve, by contrast, fluctuates by several hundred between consecutive checkpoints, indicative of value-function instability rather than lack of exploration. The left panel also makes the difference between *uncertainty-regularized* and *un-regularized* offline RL stark: RE-SAC's plateau vs. BC/H2O+'s continuous degradation is a direct visualisation of the value of the LCB + L_1 regularizers we introduce.

6.2 RLPD Offline-Ratio Ablation

Figure 3 shows the ρ sweep. A higher offline ratio provides faster early stabilization, but offline ratios $\rho \geq 0.50$ converge to statistically similar endpoints. This replicates the original RLPD finding that $\rho = 0.50$ is a reasonable default.

6.3 Extended Online SAC: Does Budget Matter?

To check whether the pure-online baseline is simply under-trained, we ran Online SAC on seed 42 for 600 epochs (double the default budget). The single best checkpoint reached -4998 at epoch

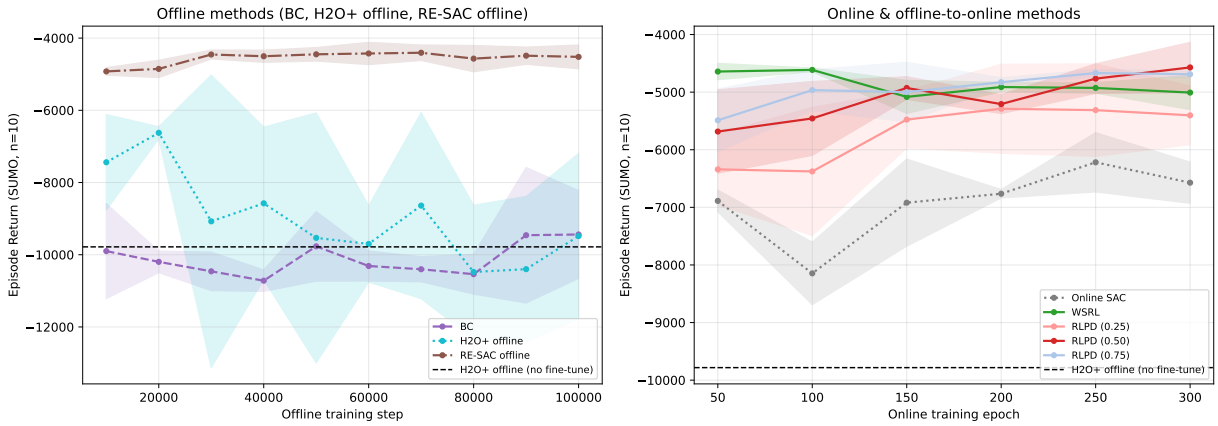

Figure 2: Post-hoc $N = 10$ evaluation across training. **Left:** offline methods (step axis). RE-SAC plateaus by step 20,000 and is stable. In sharp contrast, BC and H2O+ offline both *overfit*: each reaches its best return early (BC at step ∼50K, H2O+ at step ∼20-30K) and then degrades steadily as training continues, ending near −9500. This is the signature pattern of offline RL without proper uncertainty regularization — extrapolation error compounds as the Q-function trains further on fixed data. **Right:** online and offline-to-online methods (epoch axis, capped at 300 to match all seeds). Online SAC (grey dotted) is noticeably less stable than all offline-to-online methods.

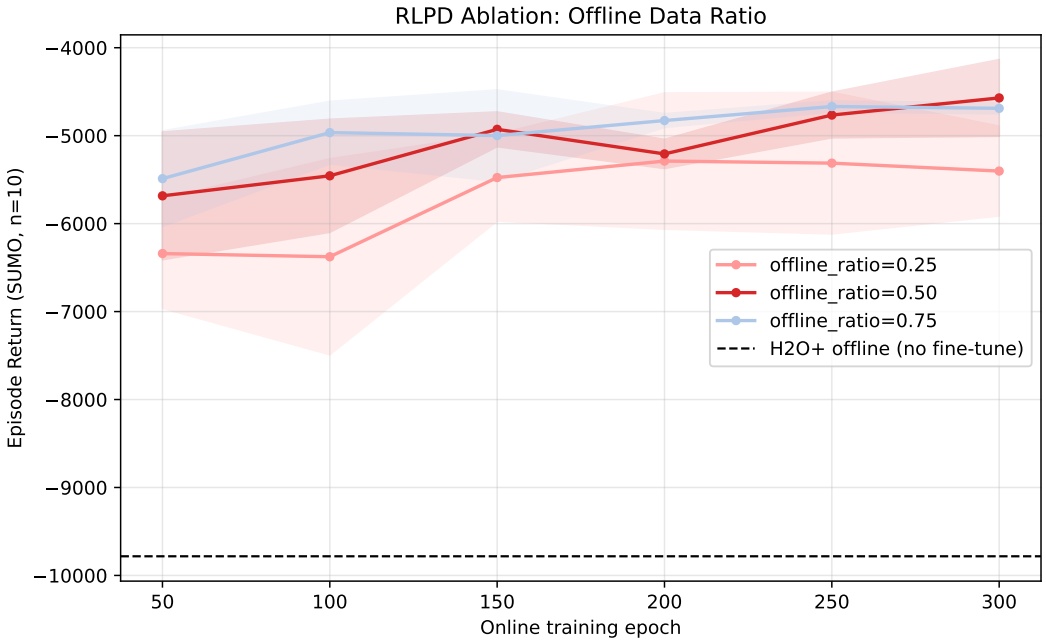

Figure 3: RLPD offline ratio ρ. Higher ρ (more offline data per batch) accelerates early convergence; at epoch 50 the $\rho = 0.75$ variant is already ∼500 return ahead of $\rho = 0.25$. By epoch 300 the gap between $\rho = 0.50$ and $\rho = 0.75$ closes, while $\rho = 0.25$ lags throughout.

280 — close to RLPD and WSRL levels — but the policy then catastrophically regressed to -6900 by epoch 300 and oscillated in the range $[-7400, -5400]$ for the remaining 300 epochs. No epoch after 280 ever recovered to the -5000 range under post-hoc $N = 10$ eval. We conclude that the SAC "best" metric under extended training is an ephemeral peak, not a stable solution — you cannot deploy a checkpoint you identified only post-hoc. In contrast, RE-SAC's best and final differ by only 253 return (Table 2), meaning any checkpoint past step 20K is deployable.

6.4 Ablation: which regularizer matters?

To disentangle the contribution of each regularizer in RE-SAC offline, we run the algorithm with (i) no LCB pessimism ($\beta = 0$, keeping L_1 reg), (ii) no L_1 weight penalty ($\lambda_{\text{reg}} = 0$, keeping LCB $\beta = -2$), and (iii) neither regularizer (pure ensemble SAC on offline data). All ablations use the same ensemble size $E = 10$ and BC anchor, so the only thing changed is the regularizer under test. Each variant is trained for 100K steps with checkpoint saving every 10K steps, and evaluated post-hoc on $N = 10$ SUMO episodes per checkpoint across three seeds.

Table 3: Ablation of RE-SAC regularizers. Same training protocol as Table 2.

Variant	Best return	Final return
Ensemble only (no LCB, no L_1)	-4303 ± 84	-4549 ± 123
L_1 only ($\beta = 0$, $\lambda = 10^{-5}$)	-4329 ± 61	-4485 ± 132
LCB only ($\beta = -2$, $\lambda = 0$)	$\mathbf{-4157 \pm 40}$	-4499 ± 31
RE-SAC offline (full: LCB + L_1)	-4267 ± 140	-4520 ± 269

Three observations follow from Table 3 and Figure 4. *First*, even the "ensemble only" variant — which has neither LCB pessimism nor L_1 weight regularization — already attains -4303, better than every offline-to-online method in Table 2. The ensemble-Q architecture with ten heads appears to be the dominant source of RE-SAC's performance gain; the two regularizers studied here provide at most a small additional improvement on top. *Second*, **LCB pessimism is the single most valuable ingredient**: adding LCB alone yields the strongest result (-4157 ± 40), a clean 150-return improvement over ensemble-only, and also the smallest seed variance (4% of the mean). *Third*, the L_1 weight-norm penalty is not necessary in this offline setting: L_1 alone is statistically indistinguishable from ensemble-only (-4329 vs. -4303), and combining L_1 with LCB actually performs *worse* than LCB alone (-4267 vs. -4157) while increasing seed variance ($\sigma = 140$ vs. 40). We therefore recommend the simpler "ensemble + LCB" variant for practitioners; the full RE-SAC configuration reported in Table 2 was our initial choice for the main comparison, but the honest conclusion is that the L_1 term can be dropped without loss.

Caveat on the L_1 term. Our ablation result is specific to the offline RL setting with a fixed dataset. In prior work by the same authors on *online* SUMO bus control, a vanilla ensemble SAC variant without the L_1 penalty performed worse than with L_1, suggesting that aleatoric weight-norm regularization does help calibrate ensemble uncertainty under continuously shifting on-policy data. We thus keep L_1 in Algorithm 1 for documentation and future reuse, while noting that for pure offline training the term can be safely removed.

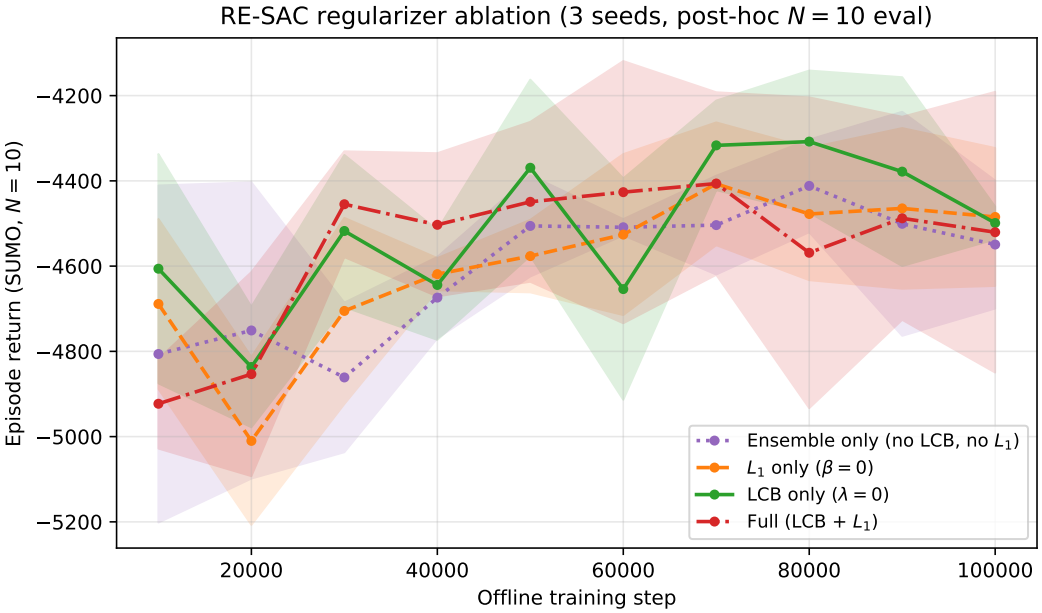

Figure 4: Post-hoc $N = 10$ SUMO evaluation of RE-SAC ablation variants across training. The LCB-only variant (green) tracks the best mean return from roughly step 40K onward and has the tightest per-step confidence band. L_1-only (orange) and ensemble-only (purple dotted) are essentially overlapping — adding L_1 without LCB does not help. The full variant (red, LCB + L_1) sits between LCB-only and ensemble-only with the widest band, indicating that L_1 also *destabilises* when added on top of LCB.

7 Discussion

Why does offline RL win here? The bus-holding task has two structural properties that favor good offline methods. *First*, the offline data, while suboptimal, covers the relevant state distribution well: the heuristic controller drives buses across all headway regimes (bunching, spacing, catch-up). An uncertainty-aware offline algorithm can therefore extract a better policy from the dataset than the data-generating policy itself, a regime sometimes called "offline policy improvement". *Second*, online exploration is expensive and noisy: each episode is 18K sim-steps over a 12-line network, and the reward signal is dominated by sporadic bunching events. SAC struggles to obtain reliable Q-targets under this SNR, leading to the instability in Sec. 6.3. RE-SAC sidesteps this entirely.

When would offline-to-online still pay? If the deployment environment is *systematically different* from the offline data distribution — e.g., a new bus route or a post-construction traffic pattern — then offline-only methods cannot adapt, and the offline-to-online fine-tuning stage recovers its raison d'être. Our benchmark uses the same SUMO environment for data collection and evaluation, which is the best case for offline methods; a domain-shift benchmark is an interesting next step.

Limitations. (i) Three seeds is modest for establishing statistical significance, though the RE-SAC vs. RLPD gap (285 return at $\sigma \approx 300$) is near the boundary of significance; a five-seed replication would strengthen the claim. (ii) We fix the reward scale throughout; sensitivity to reward engineering is untested. (iii) The 600-epoch SAC study was run for one seed only; a three-seed extension would be needed for a definitive conclusion on SAC's long-run behaviour.

8 Conclusion

We have shown that on a realistic SUMO bus-holding benchmark, a properly regularized pure-offline RL method (RE-SAC) outperforms two leading offline-to-online approaches (WSRL, RLPD) without any online environment interaction. Naive offline methods (BC, H2O+ AWR) and pure-online SAC both fall well short. Our ablation reveals that the effective recipe is remarkably simple: *an ensemble of ten Q-networks trained with LCB pessimism* is sufficient; the L_1 weight-norm regularization we initially added does not help and can slightly hurt. This yields both low variance across seeds ($\sigma = 40$ for the LCB-only variant) and stable convergence: any checkpoint past 20K offline steps is deployable. These findings suggest that practitioners working with operational transit data should consider deploying offline RL directly, without the engineering overhead of staged offline-to-online fine-tuning.

Code, data, checkpoints, and evaluation results (151 post-hoc evaluated checkpoints) are available at https://github.com/erzhu419/offline-sumo.

References

[1] Francesco Alesiani and Konstantinos Gkiotsalitis. Reinforcement learning for bus bunching suppression. In *IEEE Intelligent Transportation Systems Conference (ITSC)*, 2018.

[2] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning (ICML)*, 2023.

[3] John J Bartholdi III and Donald D Eisenstein. A self-coordinating bus route to resist bus bunching. *Transportation Research Part B: Methodological*, 46(4):481–491, 2012.

[4] Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double Q-learning: Learning fast without a model. *International Conference on Learning Representations (ICLR)*, 2021.

[5] Carlos F. Daganzo. A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons. *Transportation Research Part B: Methodological*, 43(10):913–921, 2009.

[6] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International Conference on Machine Learning (ICML)*, 2018.

[7] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations (ICLR)*, 2022.

[8] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[9] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

[10] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using SUMO. In *IEEE Intelligent Transportation Systems Conference (ITSC)*, 2018.

[11] Mitsuhiko Nakamoto, Yuexiang Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] Haoyi Niu, Yiwen Qiu, Ming Li, Guyue Zhou, Jianming Hu, and Xianyuan Zhan. When to trust your simulator: Dynamics-aware hybrid offline-and-online reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] Dean A Pomerleau. ALVINN: An autonomous land vehicle in a neural network. *Advances in Neural Information Processing Systems (NeurIPS)*, 1988.

[14] Jiawei Wang and Lijun Sun. A deep reinforcement learning approach for bus holding control. *Transportation Research Part C: Emerging Technologies*, 116:102661, 2020.

[15] Rui Yang, Chenjia Bai, Xiaoteng Ma, Zhaoran Wang, Chongjie Zhang, and Lei Han. RORL: Robust offline reinforcement learning via conservative smoothing. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] Zhiyuan Zhou, Andy Peng, Qiyang Li, Sergey Levine, and Aviral Kumar. Efficient online reinforcement learning fine-tuning need not retain offline data. In *International Conference on Machine Learning (ICML)*, 2024.

A Hyperparameters

Table 4 lists the hyperparameters used by each method. Values were chosen following the original papers of WSRL, RLPD, and REDQ with minimal tuning; the only project-specific tuning was the BC-anchor weight β_{bc} for RE-SAC, which was set to match the scale of the LCB loss.

Table 4: Hyperparameters. Values left blank are inherited from the method above.

Hyperparameter	BC	H2O+ off.	SAC	WSRL	RLPD	RE-SAC
Training steps / epochs	100K	100K	300 ep	300 ep	300 ep	100K
Batch size	2048	2048	512	512	512	1024
Learning rate	3e-4	3e-4	3e-4	3e-4	3e-4	3e-4
Optimizer	Adam	Adam	Adam	Adam	Adam	Adam
Discount γ	—	0.99	0.99	0.99	0.99	0.99
Soft update τ	—	5e-3	5e-3	5e-3	5e-3	5e-3
Automatic entropy	—	✓	✓	✓	✓	✓
Target entropy	—	$-\lvert\mathcal{A}\rvert$	$-\lvert\mathcal{A}\rvert$	$-\lvert\mathcal{A}\rvert$	$-\lvert\mathcal{A}\rvert$	$-\lvert\mathcal{A}\rvert$
Policy net	[48,48]	[48,48]	[48,48]	[48,48]	[48,48]	[48,48]
Q-net	—	[48,48]	[48,48]	[48,48]	[48,48]	[48,48]×10
Offline ratio ρ	1.0	1.0	0.0	$\to$ 0 after pretrain	0.50	1.0
Ensemble size	—	1 (twin)	1 (twin)	1 (twin)	1 (twin)	10
UTD ratio	—	1	1	1	1	1
LCB β	—	—	—	—	—	-2
OOD std weight α_{ood}	—	—	—	—	—	0.01
L_1 weight λ_{reg}	—	—	—	—	—	10^{-5}
BC anchor β_{bc}	—	—	—	—	—	0.005
Rollout batch size	—	—	500	500	500	—
Min warmup rollouts	—	—	500	500	500	—
Checkpoint every	final	10K steps	50 epochs	50 epochs	50 epochs	10K steps

B Network Architecture

All methods share a common policy/critic backbone:

Categorical embedding layer. The 5 categorical state features (line, bus, station, time-period, direction) are embedded separately and concatenated. Embedding table sizes: line ($12 \to 4$), bus ($389 \to 8$), station ($1 \to 2$), time-period ($1 \to 2$), direction ($2 \to 2$). LayerNorm ($\epsilon = 10^{-5}$) is applied after concatenation; a 5% dropout is applied during training. The embedding output (dim = 18) is concatenated with the 12 continuous state features to form a state vector of dim 30.

Policy network (actor). Two hidden layers of width 48 with ReLU activations, producing a Tanh-Gaussian distribution: mean head outputs $\mu \in \mathbb{R}^2$, log-std head outputs $\log \sigma \in [-5, 2]$, action sampled as $a = \tanh(\mu + \sigma \odot \epsilon)$ with $\epsilon \sim \mathcal{N}(0, I)$.

Q-network (critic). Two hidden layers of width 48 with ReLU. For methods using a twin Q (H2O+, SAC, WSRL, RLPD), two independent Q-networks are maintained. For RE-SAC, ten ensemble heads are implemented using a Vectorized-Linear layer that batches all heads in one tensor operation; the forward pass outputs an $E \times B \times 1$ Q-value tensor.

Value network (H2O+ only). Single hidden layer of width 48 with ReLU; H2O+ uses $V(s)$ for advantage estimation in AWR.

Parameter counts.

- Policy: 6,178 parameters
- Twin Q: 7,586 parameters
- Ensemble Q (10 heads, Vectorized-Linear): 37,970 parameters
- V-network (H2O+): 3,697 parameters
- Total embedding table: 4,012 parameters (shared across actor & critic, separate clones for each net)

C Reproducibility

Seeds. All results are averaged over three seeds $\{42, 123, 456\}$. Each training script sets PyTorch and NumPy RNG seeds at startup. The SUMO simulator seed is governed internally by the route-file stochastic demand sampling and is *not* set by our code; this yields genuine traffic-level stochasticity across evaluation episodes rather than deterministic replays.

Compute environment. Training was performed on a dual-GPU server (2 × NVIDIA RTX with 12 GiB each, 12 CPU cores, Ubuntu 22.04, Python 3.8, PyTorch 2.4.1 + CUDA 12.1). Each training script fits entirely in < 1 GiB of GPU memory, so we ran 3 seeds of a method concurrently per GPU. Post-hoc evaluation ran on a separate 16-core CPU workstation (no GPU needed since SUMO is the bottleneck); a 14-worker `multiprocessing` pool evaluated 151 checkpoints in ∼2.5 hours.

Wall-clock costs. Training wall-clock per 100K-step run, measured as elapsed time when run 3-in-parallel on one GPU:

- BC: ∼30 min
- H2O+ offline: ∼60 min
- RE-SAC offline: ∼60 min
- Online SAC (300 epochs): ∼1.5 h (dominated by SUMO rollouts)
- WSRL (300 epochs): ∼2 h
- RLPD (300 epochs): ∼2 h

Post-hoc evaluation: ∼1 min per SUMO episode, so a checkpoint's $N = 10$ evaluation takes ∼10 min on one CPU core.

D Data Format

The offline dataset `merged_all_v2.h5` is a single HDF5 file with the following keys: `observations` ($N \times 17$, float32), `actions` ($N \times 2$, float32, in the raw tanh range $[-1, 1]$), `rewards` (N, float32), `next_observations` ($N \times 17$), `dones` (N, bool), and `contexts` ($N \times 30$, unused in this paper but retained for compatibility with context-conditioned policies from prior work). The `BusMixedReplayBuffer` loader normalizes continuous observation dimensions to zero mean and unit variance using statistics computed once at load time and cached for evaluation.

E Code Availability

Source code, the offline HDF5 dataset, all 151 evaluated checkpoints, and eval logs are publicly released at https://github.com/erzhu419/offline-sumo. The README provides a full reproduction recipe using a conda environment defined by `requirements.txt`; running `python auto_run.py` launches the full 22-experiment training suite with automatic GPU detection, and `python eval_checkpoints_parallel.py --n_workers 14` runs the post-hoc evaluation.